Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

Факультет прикладной информатики

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«Мультивендорная корпоративная сеть: MikroTik CHR как vCPE»

Направление 11.03.02 «Инфокоммуникационные технологии и системы связи»

Образовательная программа «Программирование в инфокоммуникационных системах»

Дисциплина «Виртуализация сетевых функций»

Преподаватель:
Аминов Н. С.

Выполнил:
студент группы K3421
Стафеев И.А.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ **3**

1 Подготовка образов **4**

2 Топология и доступ **6**

3 Адресация **9**

4 eBGP между SONiC и MikroTik **11**

5 vCPE: NAT и межсетевой экран **14**

6 Филиал через WireGuard **17**

7 Работа через REST API и конфигурация как код **20**

8 Воспроизводимость лабораторной работы **22**

ЗАКЛЮЧЕНИЕ **24**

ВВЕДЕНИЕ

Цель работы: освоить принципы виртуализации сетевых функций (NFV) на примере построения мультивендорной корпоративной сети с использованием MikroTik CHR в роли виртуального пограничного маршрутизатора (vCPE), реализующего функции NAT, межсетевого экрана и туннелирования WireGuard.

Для достижения цели были поставлены следующие задачи:

1. развернуть мультивендорную топологию в среде Containerlab, включающую коммутатор SONiC с FRR, маршрутизаторы MikroTik CHR и Linux-хосты;
2. настроить eBGP-взаимодействие между FRR (SONiC) и RouterOS 7 для обмена маршрутами между ЦОД и головным офисом;
3. реализовать функции vCPE: NAT для выхода в интернет и межсетевой экран с политикой “запрещено всё, что не разрешено“;
4. организовать защищённое соединение между головным офисом и филиалом через туннель WireGuard и анонсировать сеть филиала в ЦОД;
5. управлять vCPE через REST API и обеспечить воспроизводимость конфигурации как кода (Infrastructure as Code).

1 Подготовка образов

Для обеспечения воспроизводимости лабораторной работы (Infrastructure as Code) был использован фреймворк `vrnetlab`, который оборачивает полноценные виртуальные машины сетевых устройств в Docker-контейнеры.

Для MikroTik CHR 7.23.8 был загружен официальный образ. Перед сборкой в `Dockerfile` были заменены зеркала Debian на архивные (`archive.debian.org`) для избежания ошибок загрузки пакетов, а также очищен кэш сборщика (рисунки 1.1, 1.2).

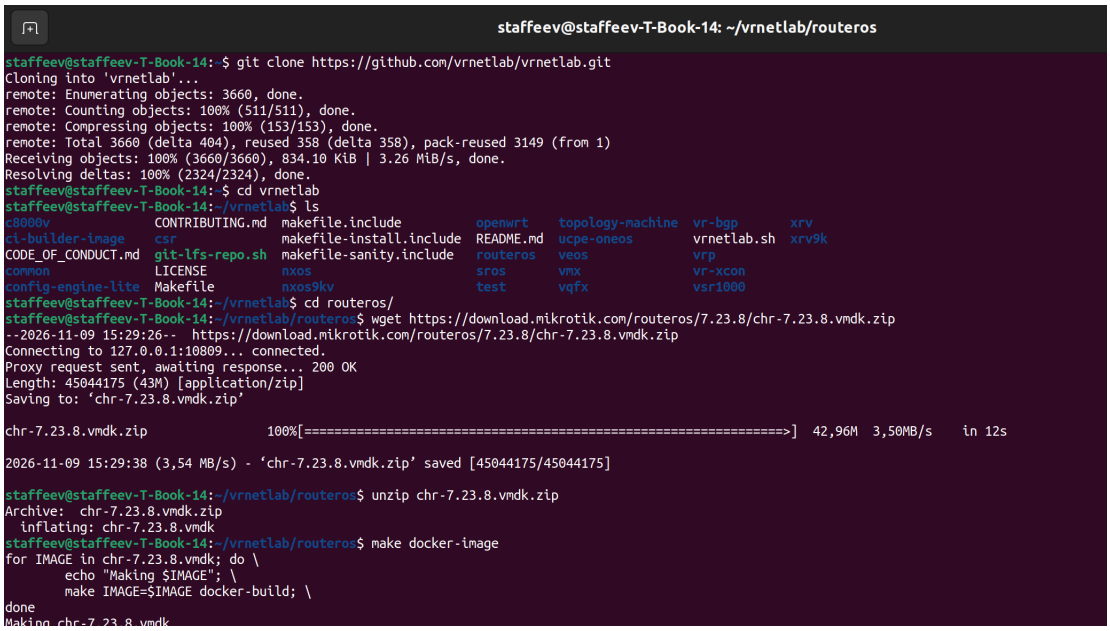


Рисунок 1.1 — Клонирование vrnetlab и загрузка образа CHR 7.23.8

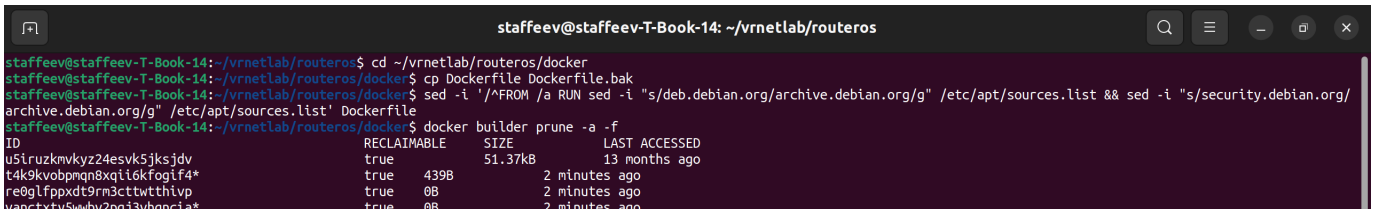


Рисунок 1.2 — Модификация Dockerfile и очистка кэша Docker

Сборка образа `vrnetlab/vr-routeros:7.23.8` прошла успешно, установив необходимые зависимости (QEMU-KVM, bridge-utils) (рисунок 1.3). Для узла SONiC собран кастомный образ `sonic-course:202511` с добавлением утилит `sudo`, `docker-cli` и `vtysh` для работы скриптов ав-

томатизации (рисунок 1.4). Для хостов использован легковесный образ `lab-tools:1.0`.

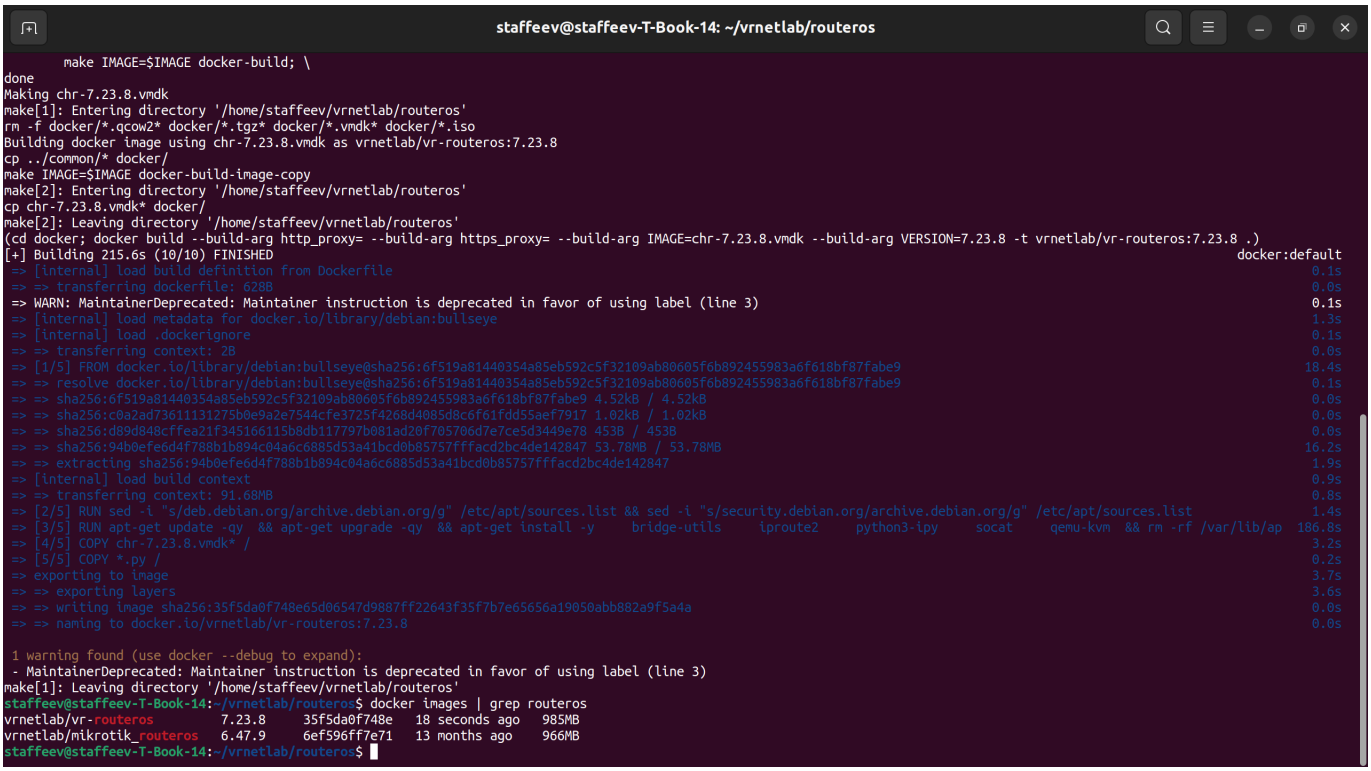


Рисунок 1.3 — Сборка образа vrnetlab/vr-routeros:7.23.8

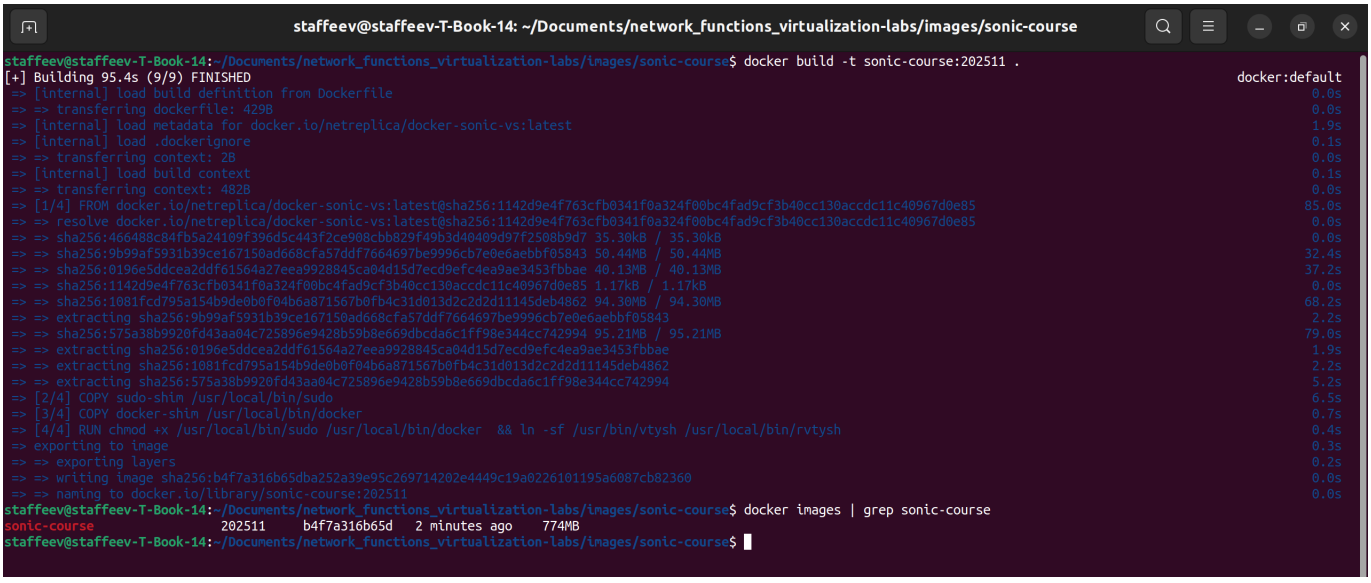


Рисунок 1.4 — Сборка образа sonic-course:202511

2 Топология и доступ

Схема развернутой мультивендорной сети представлена на рисунке 2.1. Развертывание выполнено командой `containerlab deploy`. Узлы `hq` и `br` инициализируются дольше остальных из-за загрузки полноценной ОС внутри ВМ (рисунок 2.2).

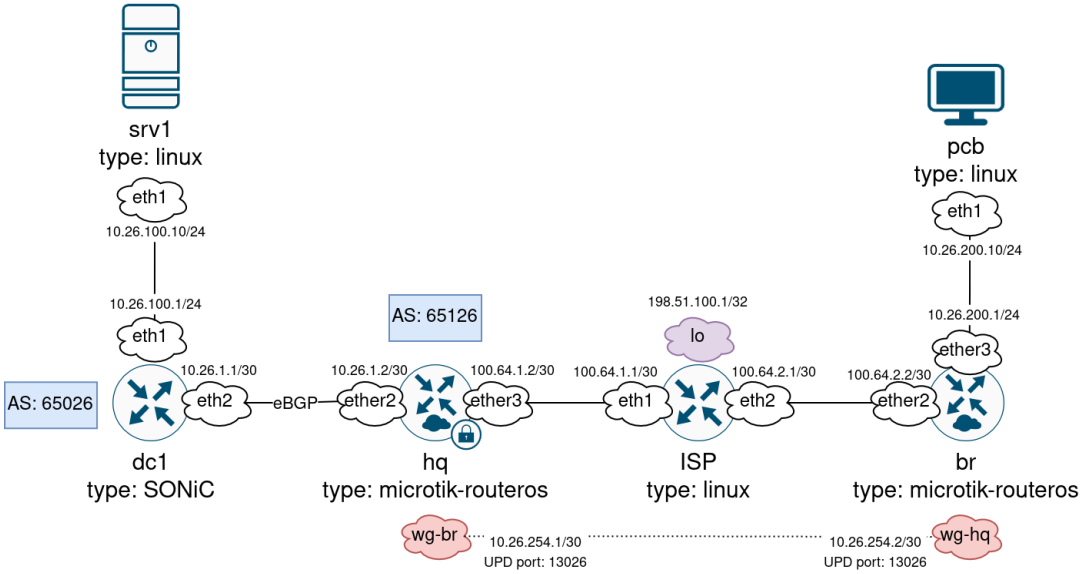


Рисунок 2.1 — Топология сети

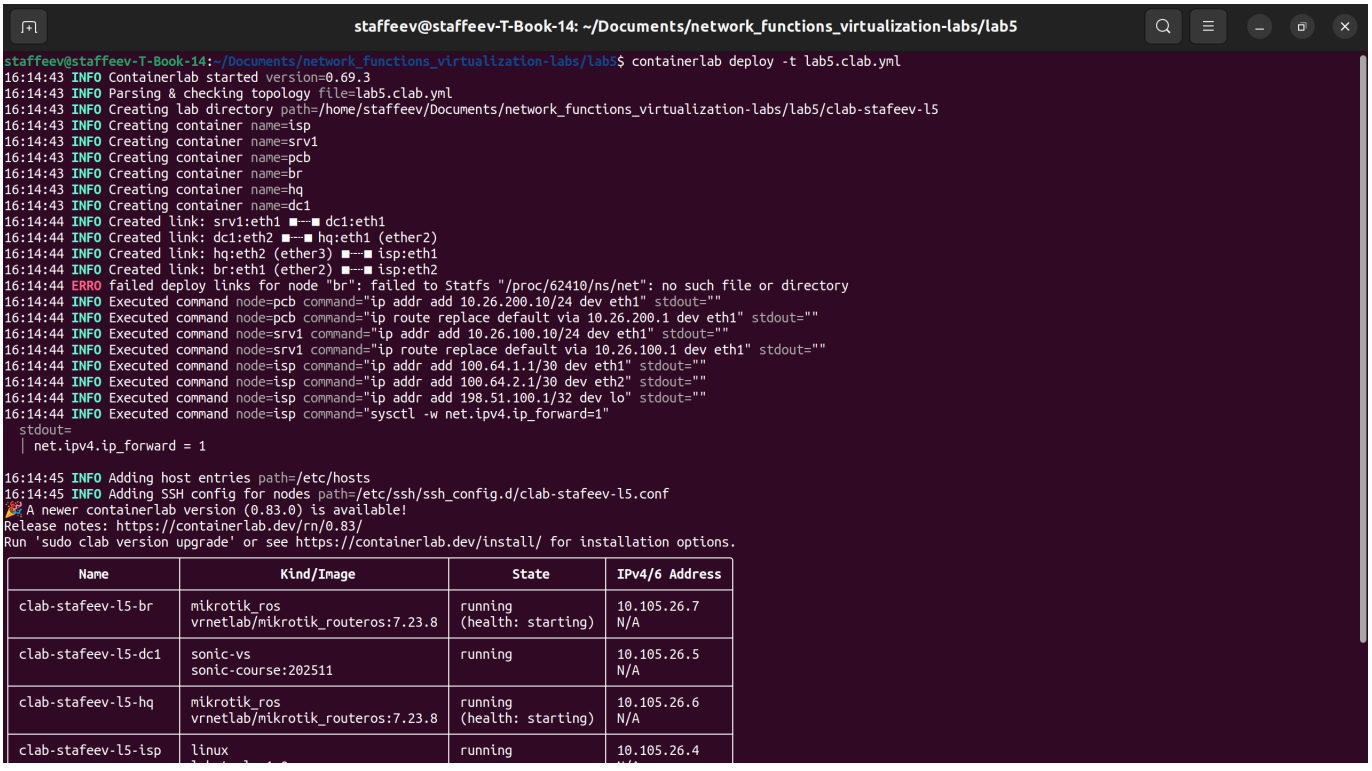


Рисунок 2.2 — Развертывание топологии в Containerlab

После загрузки выполнено первичное подключение по SSH. На узлах установлены имена `hq-stafeev` и `br-stafeev`, проверены системные ресурсы, лицензия и интерфейсы (рисунки 2.3, 2.4). Вывод подтверждает, что CHR работает как ВМ на базе QEMU/KVM.

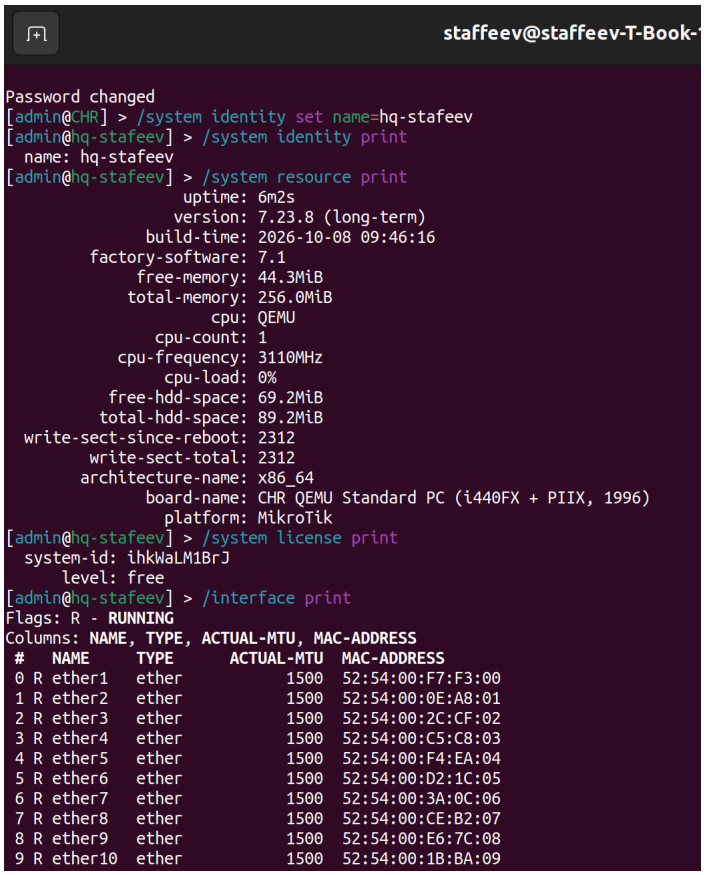


Рисунок 2.3 — Первичная проверка узла hq-stafeev

```
Password changed
[admin@CHR] > /system identity set name=br-stafeev
[admin@br-stafeev] > /system identity print
  name: br-stafeev
[admin@br-stafeev] > /system resource print
                   uptime: 8m10s
                  version: 7.23.8 (long-term)
               build-time: 2026-10-08 09:46:16
         factory-software: 7.1
              free-memory: 43.9MiB
             total-memory: 256.0MiB
                      cpu: QEMU
                cpu-count: 1
            cpu-frequency: 3110MHz
                 cpu-load: 1%
           free-hdd-space: 69.2MiB
          total-hdd-space: 89.2MiB
  write-sect-since-reboot: 2416
         write-sect-total: 2416
        architecture-name: x86_64
               board-name: CHR QEMU Standard PC (i440FX + PIIX, 1996)
                 platform: MikroTik
[admin@br-stafeev] > /system license print
  system-id: qJxx0OstbeO
      level: free
[admin@br-stafeev] > /interface print
Flags: R - RUNNING
Columns: NAME, TYPE, ACTUAL-MTU, MAC-ADDRESS
 #   NAME     TYPE      ACTUAL-MTU  MAC-ADDRESS
 0 R ether1   ether           1500  52:54:00:07:90:00
 1 R ether2   ether           1500  52:54:00:A1:07:01
 2 R ether3   ether           1500  52:54:00:40:BD:02
 3 R ether4   ether           1500  52:54:00:EA:75:03
 4 R ether5   ether           1500  52:54:00:61:30:04
 5 R ether6   ether           1500  52:54:00:C9:08:05
 6 R ether7   ether           1500  52:54:00:D5:6F:06
 7 R ether8   ether           1500  52:54:00:38:37:07
 8 R ether9   ether           1500  52:54:00:A2:D0:08
 9 R ether10  ether           1500  52:54:00:24:58:09
10 R ether11  ether           1500  52:54:00:30:31:0A
11 R ether12  ether           1500  52:54:00:56:FE:0B
12 R ether13  ether           1500  52:54:00:43:27:0C
```

Рисунок 2.4 — Первичная проверка узла br-stafeev

Почему CHR требует аппаратной виртуализации, а SONiC — нет?

MikroTik CHR — это полноценная ОС, упакованная в образ ВМ (.vmdk), для запуска которой необходим гипервизор (QEMU/KVM), требующий поддержки флагов VMX/SVM на хосте. SONiC в данной работе использует контейнерный образ (`docker-sonic-vs`), который работает непосредственно на ядре хостовой ОС Linux через механизмы namespaces и cgroups, не эмулируя аппаратное обеспечение.

Чем ограничена бесплатная лицензия и как это скажется на измерениях скорости?

Лицензия `free` ограничивает исходящую скорость каждого интерфейса CHR до 1 Мбит/с. Любые замеры пропускной способности (например, через `iperf3`) будут искусственно ограничены этим порогом, что делает невозможным тестирование реальной производительности на высоких скоростях. Для проверки связности и маршрутизации этого достаточно.

3 Адресация

На пограничных маршрутизаторах `hq` и `br` настроена базовая IP-адресация LAN/WAN интерфейсов и статические маршруты по умолчанию через шлюзы провайдера (`100.64.1.1` и `100.64.2.1` соответственно) (рисунок 3.1).

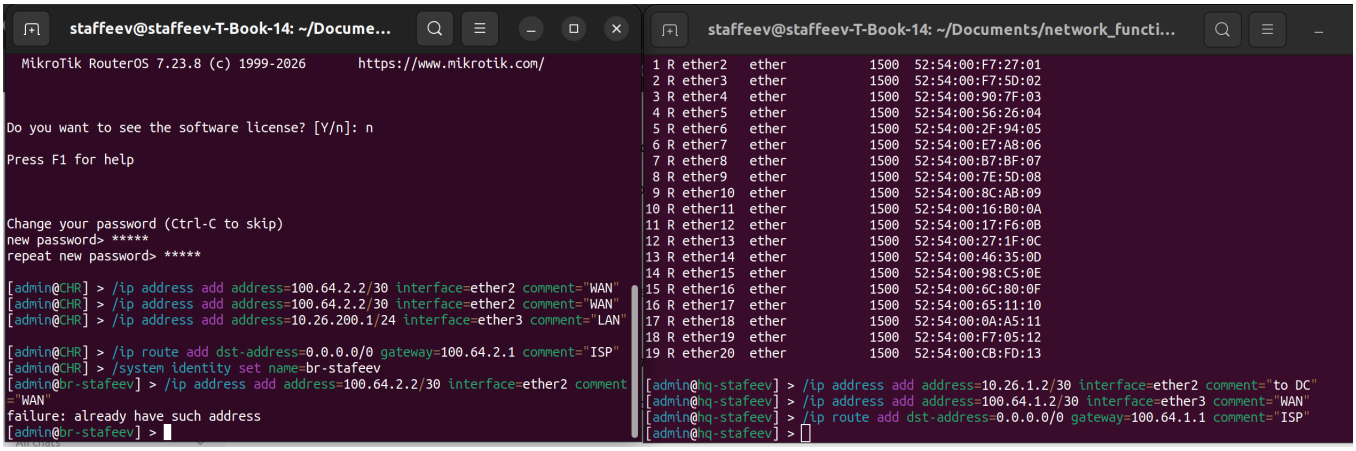


Рисунок 3.1 — Настройка адресации и маршрутов на hq и br

На узле SONiC `dc1` запущены демоны FRR, подняты интерфейсы `Ethernet0` (`10.26.100.1/24`) и `Ethernet4` (`10.26.1.1/30`). Проверка связности с `hq` (`10.26.1.2`) показала 0% потерь (рисунок 3.2).

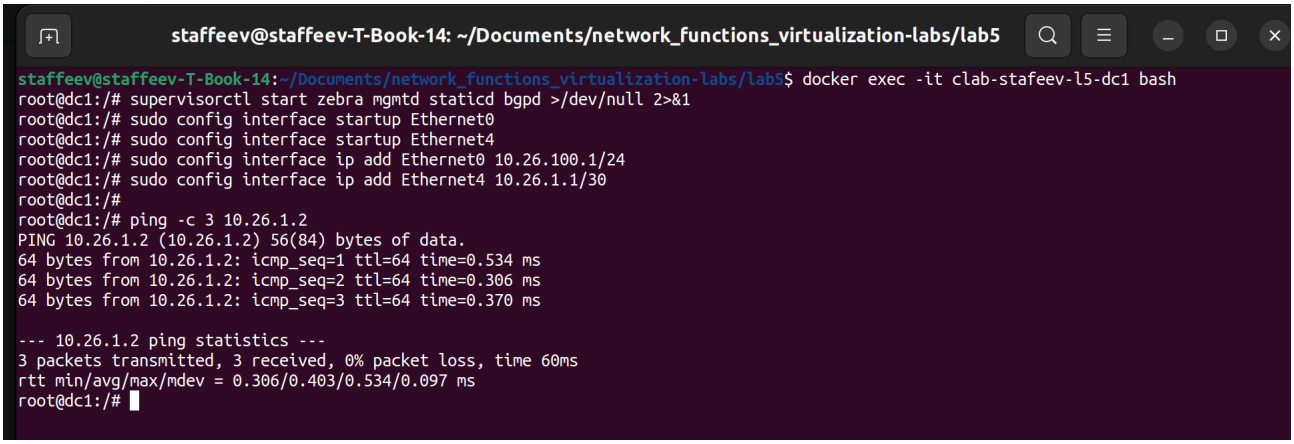


Рисунок 3.2 — Настройка адресации на dc1 и проверка связности

Дополнительно подтверждена корректная маршрутизация до провайдера: пинги с `hq` до `100.64.1.1` и с `br` до `100.64.2.1` проходят без потерь (рисунок 3.3).

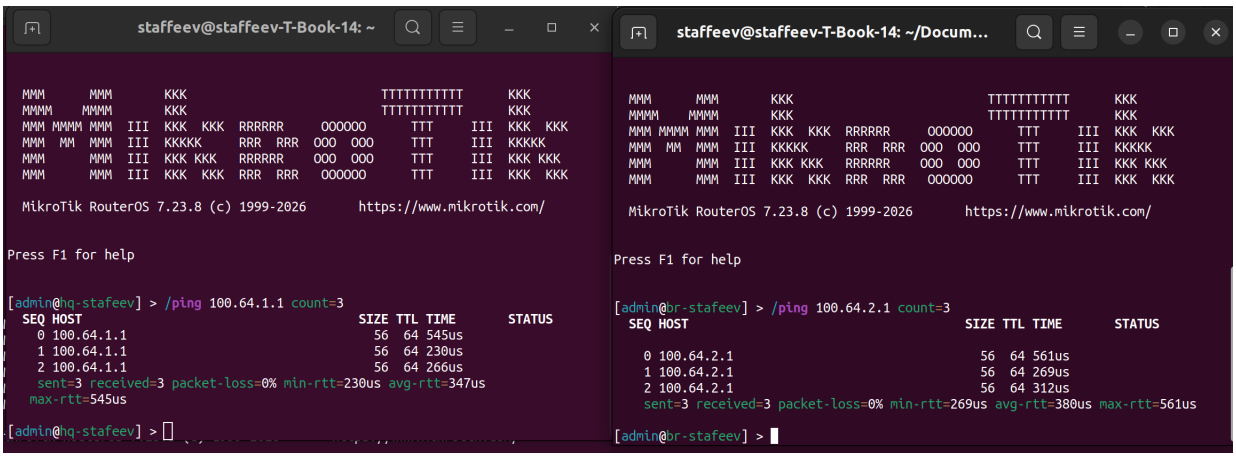


Рисунок 3.3 — Проверка связности с узлом провайдера

4 eBGP между SONiC и MikroTik

Для обмена маршрутами между ЦОД и головным офисом настроен eBGP. Критически важным первым шагом на `dc1` стало удаление маршрута по умолчанию, полученного через management-интерфейс (`ip route del default dev eth0`) (рисунок 4.1).

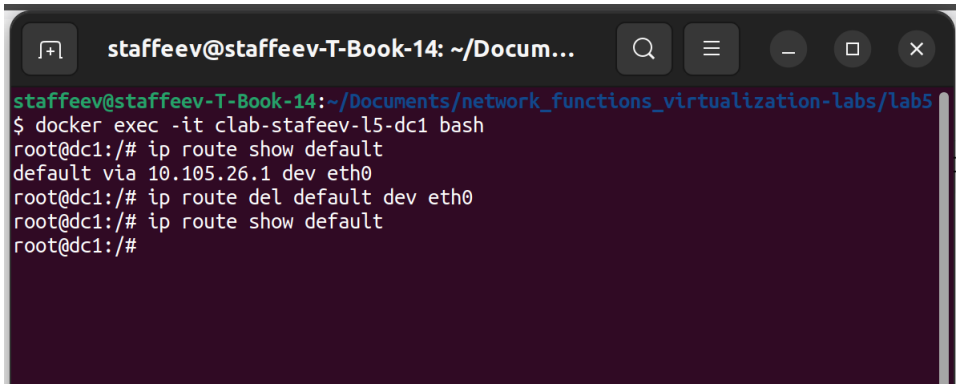


Рисунок 4.1 — Удаление management-маршрута по умолчанию на dc1

Почему без этого шага маршрут по умолчанию, полученный по BGP, не будет использоваться?

Маршрутизатор выбирает путь с наименьшим административным расстоянием (AD). У статических или DHCP-маршрутов ядра (`K`) AD равно 0 или 1. У маршрутов eBGP (`B`) AD равно 20. Если не удалить management-маршрут, трафик будет идти через него, игнорируя корректный BGP-путь через `hq`.

Далее на `dc1` через `vtysh` применена конфигурация FRR: задан AS 65026, сосед `10.26.1.2` (AS 65126) и анонсирована сеть `10.26.100.0/24` (рисунок 4.2). На `hq` создан BGP-инстанс и соединение с параметром `output.default-originate=if-installed` (рисунок 4.3).

```
dc1# configure terminal
dc1(config)# router bgp 65026
dc1(config-router)# bgp router-id 10.26.1.1
dc1(config-router)#  no bgp ebgp-requires-policy
dc1(config-router)#  neighbor 10.26.1.2 remote-as 65126
dc1(config-router)# address-family ipv4 unicast
dc1(config-router-af)# network 10.26.100.0/24
dc1(config-router-af)# exit-address-family
dc1(config-router)# end
dc1# write memory
Note: this version of vtysh never writes vtysh.conf

Warning: attempting direct configuration write without watchfrr.
File permissions and ownership may be incorrect, or write may fail.

Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
dc1# exit
root@dc1:/# vtysh -c "show bgp summary"

IPv4 Unicast Summary:
BGP router identifier 10.26.1.1, local AS number 65026 vrf-id 0
BGP table version 1
RIB entries 1, using 192 bytes of memory
Peers 1, using 21 KiB of memory

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt
10.26.1.2       4      65126         0         0        0    0    0    never       Active        0

Total number of neighbors 1
root@dc1:/# vtysh -c "show ip route"
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, D - SHARP,
       F - PBR, f - OpenFabric,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup

C>* 10.26.1.0/30 is directly connected, Ethernet4, 01:14:20
C>* 10.26.100.0/24 is directly connected, Ethernet0, 01:14:22
C>* 10.105.26.0/24 is directly connected, eth0, 01:16:04
root@dc1:/#
```

Рисунок 4.2 — Применение конфигурации BGP на узле dc1

```
[admin@hq-stafeev] > /routing/bgp/instance add name=hq as=65126
[admin@hq-stafeev] > /routing/bgp/connection add name=to-dc instance=hq remote.address=10.26.1.1 remote.as=65026 local.role=ebgp output.default-originate=if-installed
[admin@hq-stafeev] > /routing/bgp/session print
Flags: E - ESTABLISHED
 0 E name="to-dc-1" instance=hq
     remote.address=10.26.1.1 .as=65026 .id=10.26.1.1 .capabilities=mp,rr,gr,as4,ap,fqdn .afi=ip .messages=3 .bytes=127 .gr-time=120 .eor=ip
     local.address=10.26.1.2 .as=65126 .id=172.31.255.30 .cluster-id=172.31.255.30 .capabilities=mp,rr,enhe,gr,as4 .afi=ip .messages=2 .bytes=64 .eor=""
     output.procid=20 .default-originate=if-installed
     input.procid=20 ebgp
     hold-time=3m keepalive-time=1m uptime=3s290ms last-started=2026-11-09 23:10:13 prefix-count=0
[admin@hq-stafeev] > /ip route print where bgp
Flags: D - DYNAMIC; A - ACTIVE; b - BGP
Columns: DST-ADDRESS, GATEWAY, ROUTING-TABLE, DISTANCE
    DST-ADDRESS     GATEWAY    ROUTING-TABLE  DISTANCE
DAb 10.26.100.0/24  10.26.1.1  main                20
[admin@hq-stafeev] >
```

Рисунок 4.3 — Настройка BGP на узле hq

Почему `output.default-originate=if-installed` надёжнее, чем `always`?

Параметр **`if-installed`** предписывает анонсировать маршрут `0.0.0.0/0` только если он реально присутствует в таблице маршрутизации **`hq`**. При обрыве канала до провайдера этот маршрут исчезнет, и анонс прекратится. Режим **`always`** продолжил бы анонсировать маршрут, создав «чёрную дыру» (black hole), так как **`dc1`** отправлял бы трафик в никуда.

Итоговая проверка на `dc1` подтверждает успешное соседство (статус `Estab`) и появление активного BGP-маршрута `B>* 0.0.0.0/0 [20/0] via 10.26.1.2` (рисунок 4.4).

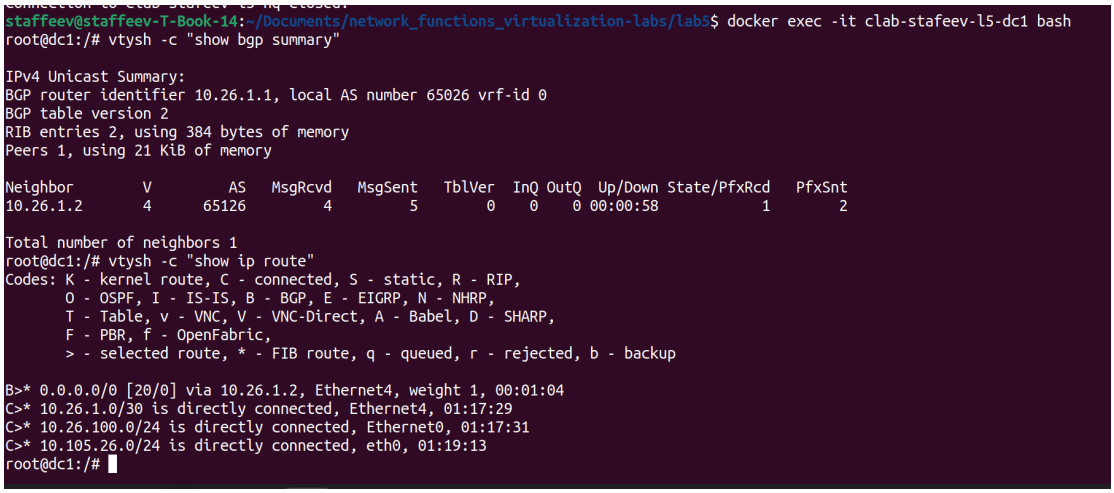


Рисунок 4.4 — Проверка BGP-сессии и таблицы маршрутизации на dc1

5 vCPE: NAT и межсетевой экран

На маршрутизаторе `hq` настроен NAT для выхода в интернет: добавлено правило `masquerade` в цепочке `srcnat` для интерфейса `ether3` (WAN). Проверка показала успешный доступ из ЦОД к интернет-ресурсу: HTTP-запрос к `198.51.100.1` вернул код 200. Логи провайдера подтверждают, что запросы приходят с адреса `100.64.1.2` (внешний адрес `hq`), а не с внутреннего `10.26.100.10` (рисунок 5.1).

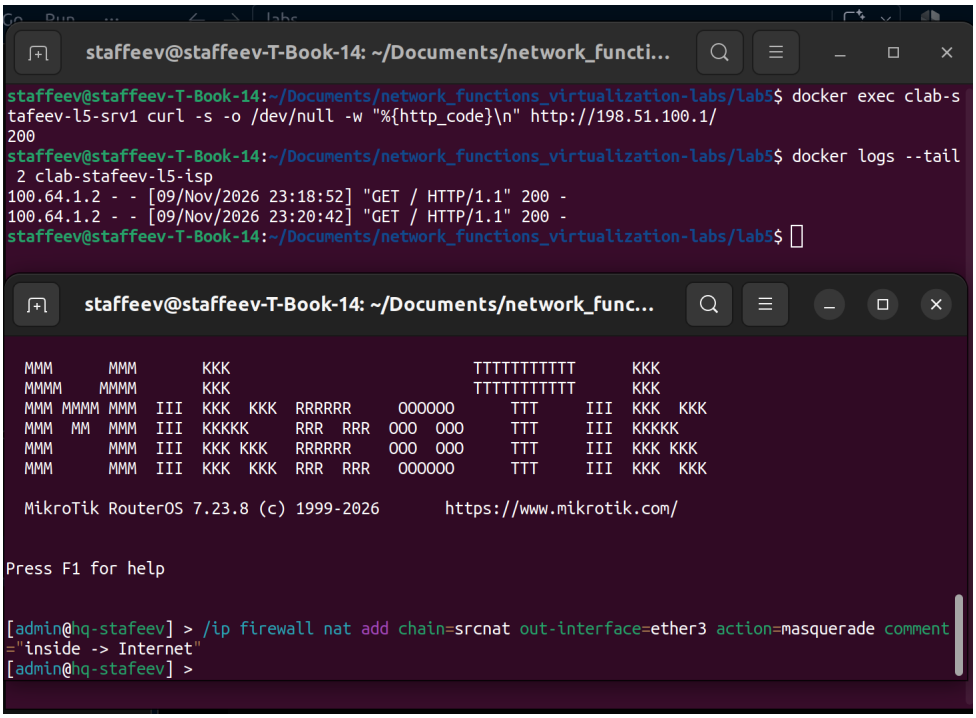


Рисунок 5.1 — Настройка NAT на hq и проверка доступа в интернет

Реализована политика межсетевого экрана “запрещено всё, что не разрешено“. На `hq` добавлены правила фильтрации:

- `input`: разрешение established/related, ICMP, management-трафика (ether1), BGP (ether2, порт 179); запрет остального трафика с WAN (ether3);
- `forward`: разрешение established/related и трафика из ЦОД в интернет (ether2 → ether3); запрет всего остального (default deny) (рисунок 5.2).

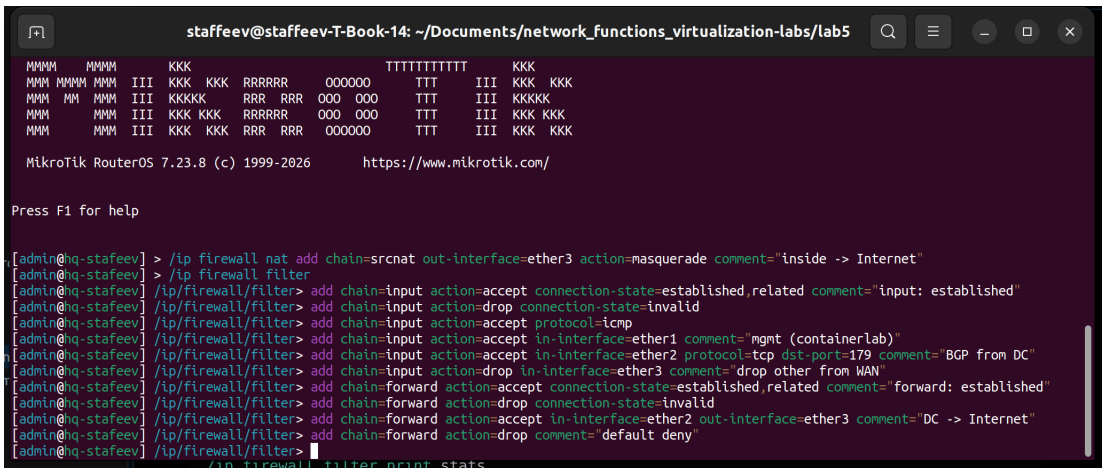


Рисунок 5.2 — Настройка межсетевого экрана на hq

Проверка политики: HTTP-запрос из ЦОД проходит успешно (код 200), тогда как ping от провайдера к серверу ЦОД (`10.26.100.10`) блокируется (100% потерь). Статистика фаервола показывает, что правило `default deny` отбросило 2 пакета (рисунок 5.3).

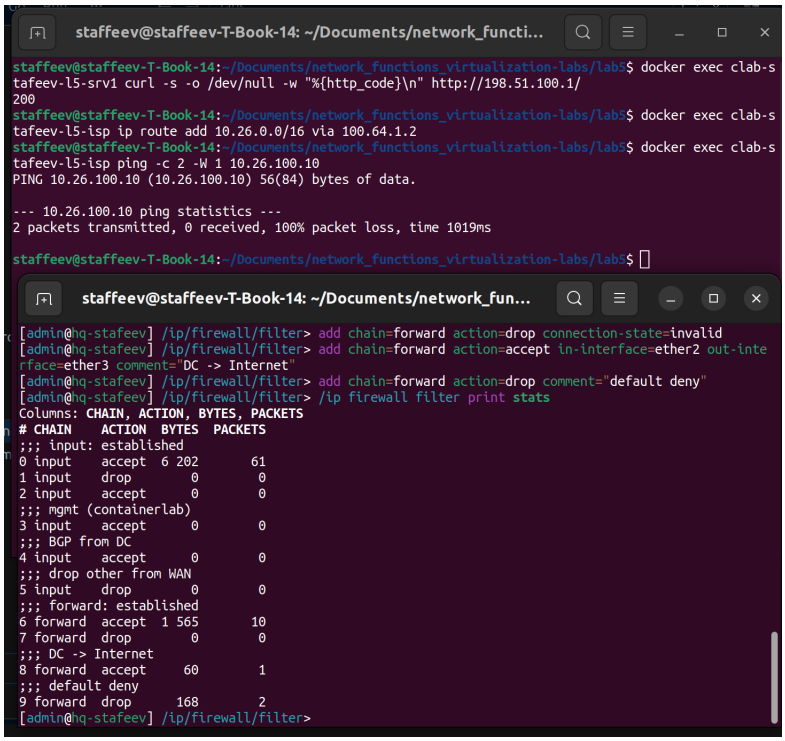


Рисунок 5.3 — Проверка работы межсетевого экрана

Почему ответы `198.51.100.1 -> srv1` проходят, хотя явного разрешающего правила для направления `ether3 -> ether2` нет?

Правило `chain=forward action=accept connection-state=established,rel` разрешает весь обратный трафик, который является частью уже установленных соединений. Когда `srv1` инициирует соединение с интернетом (исходящий трафик `ether2 -> ether3`), фаервол запоминает это соединение в таблице состояний (state table). Ответный трафик от `198.51.100.1` помечается как `established` или `related`, и первое правило в chain `forward` его пропускает. Это стандартное поведение stateful firewall — нет необходимости явно разрешать каждый обратный поток.

6 Филиал через WireGuard

Для организации защищённого соединения между головным офисом и филиалом настроен туннель WireGuard. На обоих узлах (`hq` и `br`) создан интерфейс `wg` с портом прослушивания `13026`, сгенерированы пары ключей. На `hq` добавлен пир с публичным ключом `br`, разрешёнными адресами `10.26.254.2/32` и `10.26.200.0/24` (сеть филиала). На `br` настроен пир с ключом `hq` и разрешёнными адресами `10.26.254.1/32` и `10.26.100.0/24` (сеть ЦОД) (рисунок 6.1).

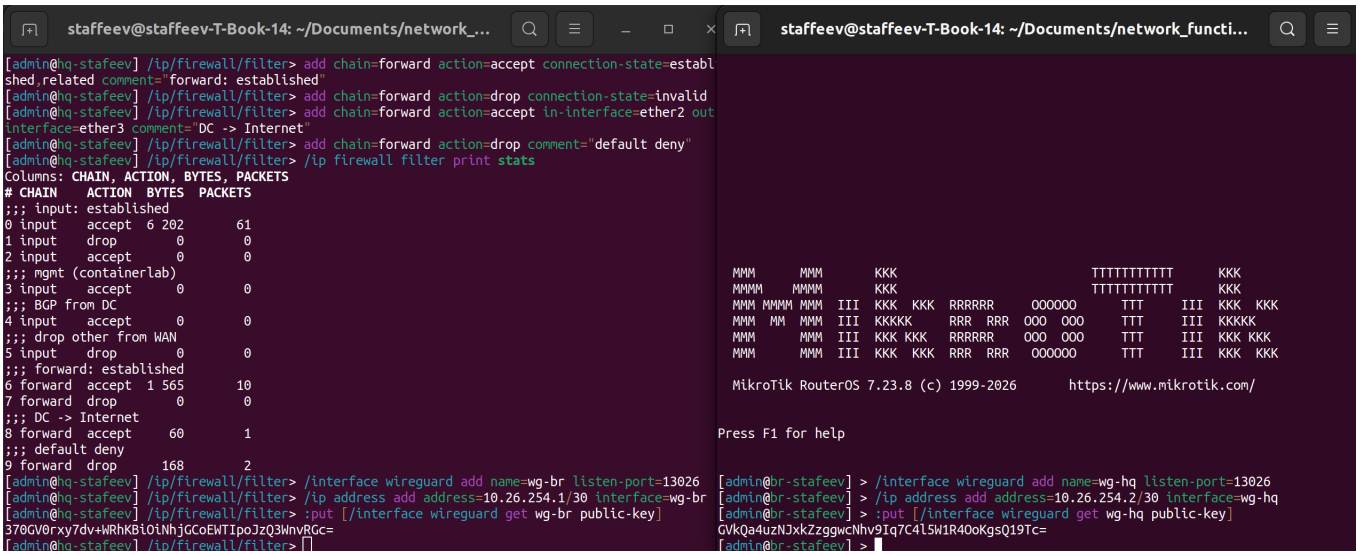


Рисунок 6.1 — Создание интерфейсов WireGuard и обмен публичными ключами

Добавлены статические маршруты: на `hq` сеть филиала `10.26.200.0/24` направлена через `wg-br`, на `br` сеть ЦОД `10.26.100.0/24` — через `wg-hq`. В межсетевой экран `hq` добавлены правила: разрешение UDP-порта 13026 для WireGuard (перед правилом `drop other from WAN`) и разрешение forward-трафика между `wg-br` и `ether2`. Сеть филиала `10.26.200.0/24` добавлена в address-list `to-dc` и анонсирована в BGP (рисунок 6.2).

```
[admin@hq-stafeev] > /interface wireguard peers add interface=wg-br public-key="GVkQa4uzNJxkZzggwcNhv9Iq7C4l5W1R4OoKgsQ19Tc=" endpoint-address=100.64.2.2 endpoint-port=13026 allowe
d-address=10.26.254.2/32,10.26.200.0/24 persistent-keepalive=25s
[admin@hq-stafeev] > /ip route add dst-address=10.26.200.0/24 gateway=wg-br
[admin@hq-stafeev] > /ip firewall filter add chain=input action=accept protocol=udp dst-port=13026 comment="WireGuard" place-before=[find comment="drop other from WAN"]
[admin@hq-stafeev] > /ip firewall filter add chain=forward action=accept in-interface=wg-br out-interface=ether2 comment="branch -> DC" place-before=[find comment="default deny"]
[admin@hq-stafeev] > /ip firewall filter add chain=forward action=accept in-interface=ether2 out-interface=wg-br comment="DC -> branch" place-before=[find comment="default deny"]
[admin@hq-stafeev] > /ip firewall filter print
Flags: X - DISABLED, I - INVALID; D - DYNAMIC
 0    ;;; input: established
      chain=input action=accept connection-state=established,related

 1    chain=input action=drop connection-state=invalid

 2    chain=input action=accept protocol=icmp

 3    ;;; mgmt (containerlab)
      chain=input action=accept in-interface=ether1

 4    ;;; BGP from DC
      chain=input action=accept protocol=tcp in-interface=ether2 dst-port=179

 5    ;;; WireGuard
      chain=input action=accept protocol=udp dst-port=13026

 6    ;;; drop other from WAN
      chain=input action=drop in-interface=ether3

 7    ;;; forward: established
      chain=forward action=accept connection-state=established,related

 8    chain=forward action=drop connection-state=invalid

 9    ;;; DC -> Internet
      chain=forward action=accept in-interface=ether2 out-interface=ether3

10    ;;; branch -> DC
      chain=forward action=accept in-interface=wg-br out-interface=ether2

[admin@hq-stafeev] > :put [/interface wireguard get wg-br public-key]
370GV0rxy7dv+WRhKBiOiNhjGCoEWTIpoJzQ3WnvRGc=
[admin@hq-stafeev] > /ip firewall address-list add list=to-dc address=10.26.200.0/24
[admin@hq-stafeev] > /routing/bgp/connection set to-dc output.network=to-dc
[admin@hq-stafeev] >
```

Рисунок 6.2 — Настройка пиров, маршрутов и анонсирование филиала в BGP на hq

На `br` настроен NAT (`masquerade`) для выхода в интернет через WAN-интерфейс `ether2` (рисунок 6.3).

```
[admin@br-stafeev] > /interface wireguard add name=wg-hq listen-port=13026
[admin@br-stafeev] > /ip address add address=10.26.254.2/30 interface=wg-hq
[admin@br-stafeev] > :put [/interface wireguard get wg-hq public-key]
GVkQa4uzNJxkZzggwcNhv9Iq7C4l5W1R4OoKgsQ19Tc=
[admin@br-stafeev] > /interface wireguard peers add interface=wg-hq public-key="370GV0rxy7dv+WRhKBiOiNhjGCoEWTIpoJzQ3WnvRGc=" endpoint-address=100.64.1.2 endpoint-port=13026 allowe
d-address=10.26.254.1/32,10.26.100.0/24 persistent-keepalive=25s
[admin@br-stafeev] > /ip route add dst-address=10.26.100.0/24 gateway=wg-hq
[admin@br-stafeev] > /ip firewall nat add chain=srcnat out-interface=ether2 action=masquerade comment="branch -> Internet"
[admin@br-stafeev] >
```

Рисунок 6.3 — Настройка пиров, маршрутов и NAT на br

Проверка показала успешную работу туннеля: на `dc1` появился BGP-маршрут до `10.26.200.0/24` через `10.26.1.2`. Ping с рабочей станции филиала (`pcb`, `10.26.200.10`) до сервера ЦОД (`srv1`, `10.26.100.10`) прошёл без потерь (3 пакета, 0% потерь). Анализ трафика на узле провайдера (`tcpdump` на `eth2`) показал только UDP-пакеты между `100.64.1.2` и `100.64.2.2` на порт `13026`. Статус пиров WireGuard на обоих концах — `last-handshake` активен, счётчики `rx/tx` увеличиваются (рисунок 6.4).

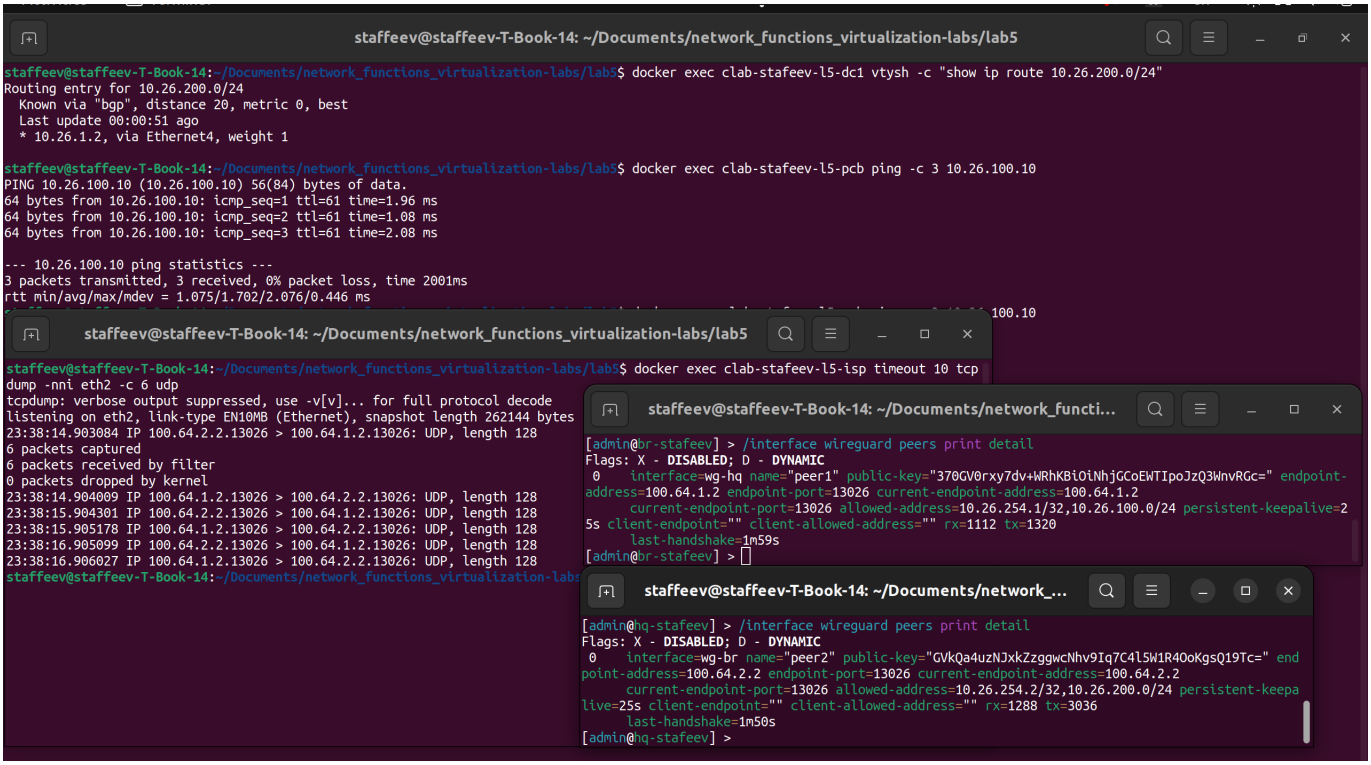


Рисунок 6.4 — Проверка связности филиала с ЦОД и анализ трафика на провайдере

Какие адреса и порты видит провайдер? Видны ли ему адреса `10.N.x.x`?

Провайдер видит только внешние IP-адреса WAN-интерфейсов: `100.64.1.2` (`hq`) и `100.64.2.2` (`br`), а также UDP-порт `13026`, используемый WireGuard для инкапсуляции трафика. Внутренние адреса сети предприятия (`10.26.x.x`) *не видны* провайдеру, так как весь трафик между `hq` и `br` инкапсулируется в зашифрованный туннель WireGuard. В выводе `tcpdump` отсутствуют какие-либо упоминания о `10.26.x.x` — только зашифрованные UDP-пакеты между публичными адресами.

7 Работа через REST API и конфигурация как код

Для управления маршрутизатором `hq` через REST API был включён встроенный веб-сервер RouterOS командой `/ip service set www disabled=no`. Разработан отдельный скрипт `rest-demo.sh`, автоматизирующий выполнение CRUD-операций: GET-запросы для чтения состояния (`/system/identity`, `/ip/address`, `/routing/bgp/session`), PUT для создания записи в address-list, PATCH для её изменения и DELETE для удаления. Скрипт выводит результаты в формате JSON, что удобно для интеграции с системами автоматизации (рисунок 7.1).

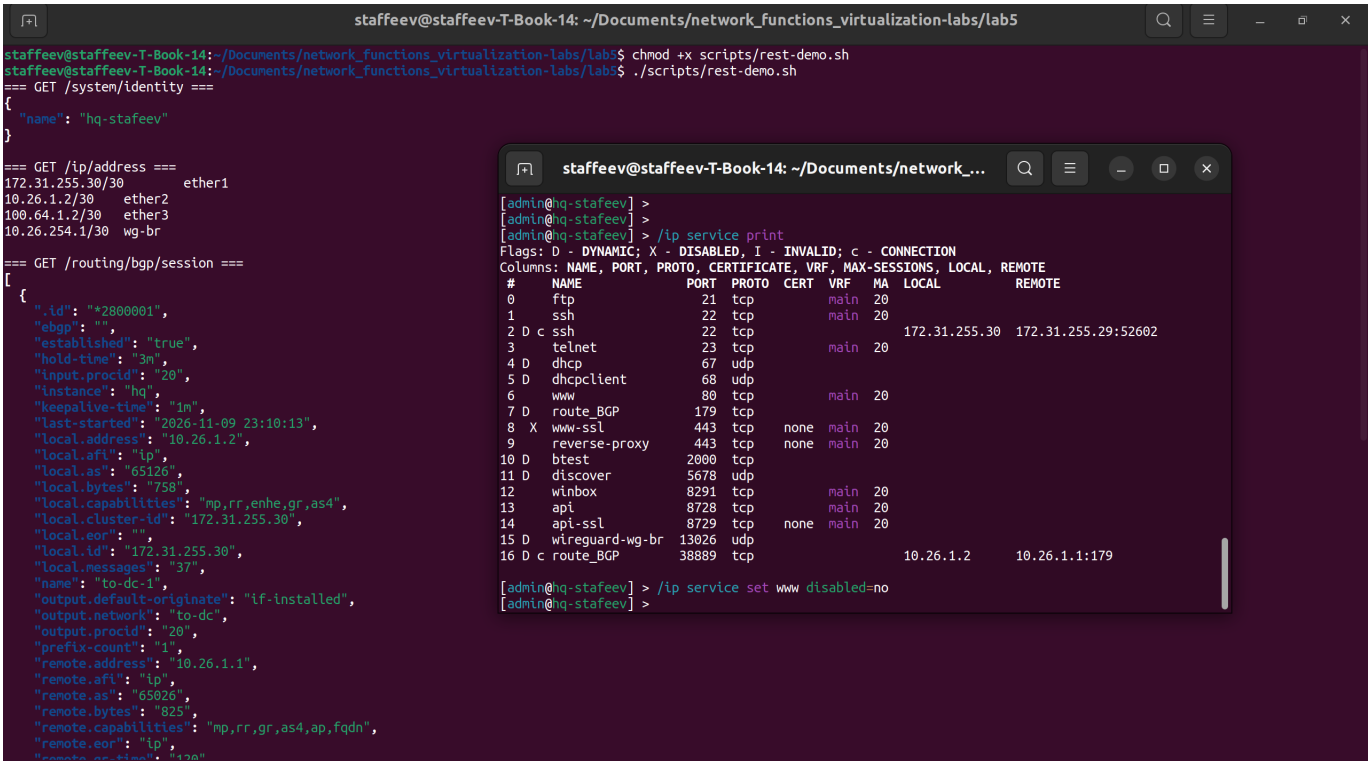


Рисунок 7.1 — Включение REST API и выполнение скрипта rest-demo.sh

Для фиксации состояния сети в системе контроля версий выполнена выгрузка конфигураций всех узлов. С MikroTik конфигурация экспортирована командой `/export terse` через SSH, с SONiC — командой `vtysh -c "show running-config"`. Полученные файлы (`hq.rsc`, `br.rsc`, `dc1-frr.conf`) сохранены в каталоге `configs/` (рисунок 7.2).

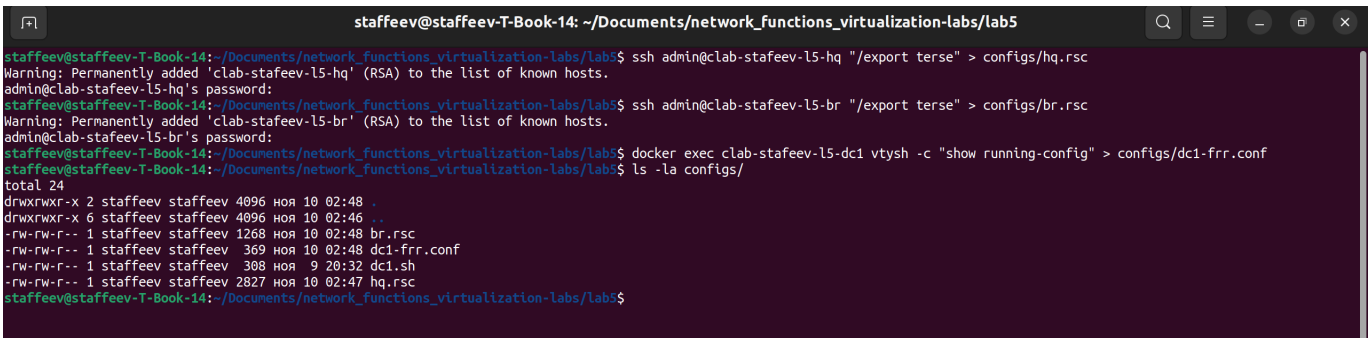


Рисунок 7.2 — Выгрузка конфигураций узлов в каталог configs/

Почему в экспорт по умолчанию не попадает приватный ключ WireGuard?

Команда `/export terse` экспортирует только конфигурационные параметры, не содержащие чувствительных данных (пароли, ключи, сертификаты). Приватный ключ WireGuard является секретом и по умолчанию исключается из экспорта для предотвращения случайной утечки. Для его выгрузки требуется явное указание флага `show-sensitive`, что создаёт риск попадания секрета в открытый доступ.

Как правильно хранить такие секреты в репозитории?

Приватные ключи и пароли никогда не должны храниться в Git-репозитории в открытом виде. Корректные подходы:

- Использование менеджеров секретов (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets);
- Хранение секретов в переменных окружения CI/CD-пайплайна с инъекцией при деплое;
- Применение placeholder'ов в конфигурационных файлах (например, `$WG_PRIVATE_KEY`), которые заполняются из защищённого хранилища при развёртывании;
- Добавление файлов с секретами в `.gitignore` для исключения из коммита;
- Генерация ключей автоматически при старте устройства (как реализовано в скрипте `deploy.sh` для обеспечения воспроизводимости).

8 Воспроизводимость лабораторной работы

Для обеспечения полной воспроизводимости лабораторной работы реализован подход Infrastructure as Code. Конфигурация сети описана декларативно в файле `lab5.clab.yml`, где для узлов MikroTik CHR задан параметр `startup-config`, автоматически применяющий конфигурацию при загрузке, а для узла SONiC `dc1` через механизм `binds` и `exec` монтируется и выполняется скрипт `apply.sh`, настраивающий интерфейсы, удаляющий management-маршрут и загружающий конфигурацию BGP в FRR.

Разработан оркестратор `deploy.sh`, автоматизирующий полный цикл развёртывания: уничтожение предыдущей топологии, деплой, ожидание загрузки узлов, динамическую настройку WireGuard с извлечением свежих публичных ключей, ожидание установления BGP-сессии, проверку связности и запуск REST API тестов (рисунок 8.1).



Рисунок 8.1 — Скрипт deploy.sh для автоматического развёртывания

Ключевой особенностью является динамическая настройка WireGuard: поскольку эмулятор `vrnetlab` при каждом старте генерирует новую пару ключей для интерфейсов MikroTik, скрипт считывает актуальные публичные ключи через SSH и добавляет пиры с корректными значениями, что обеспечивает успешный handshake туннеля.

Результат выполнения скрипта показан на рисунке 8.2: топология развёрнута, конфигурации применены, BGP-сессия установлена, пинг между филиалом и ЦОД проходит без потерь, REST API тесты выполнены успешно.

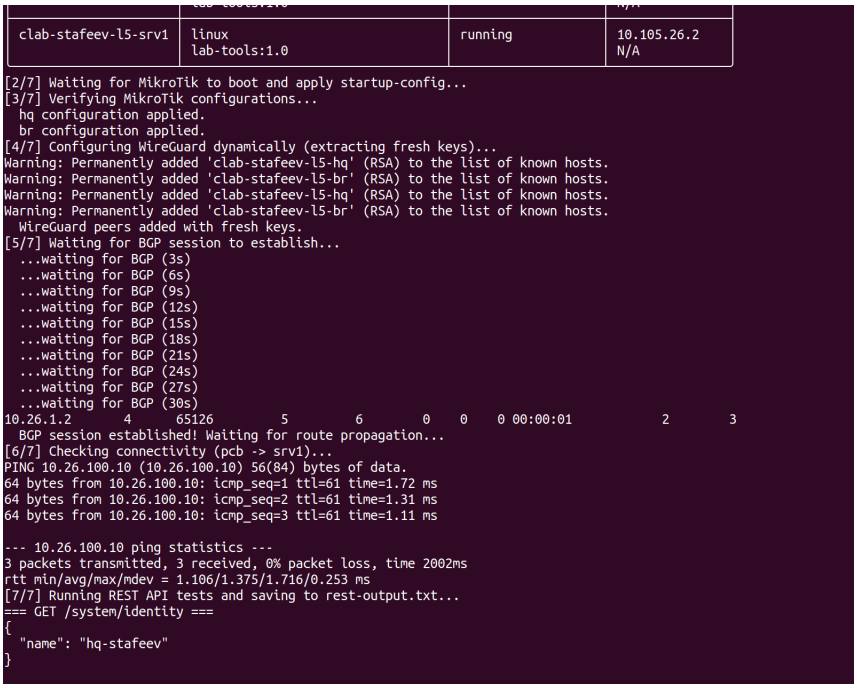


Рисунок 8.2 — Вывод скрипта deploy.sh: успешное развёртывание и проверка

Такой подход гарантирует, что лабораторная работа может быть воспроизведена с нуля одной командой `./deploy.sh` на любой системе с установленными Docker и Containerlab, без необходимости ручного ввода конфигураций.

ЗАКЛЮЧЕНИЕ

Финальный артефакт работы продемонстрирован на рисунке 8.3.

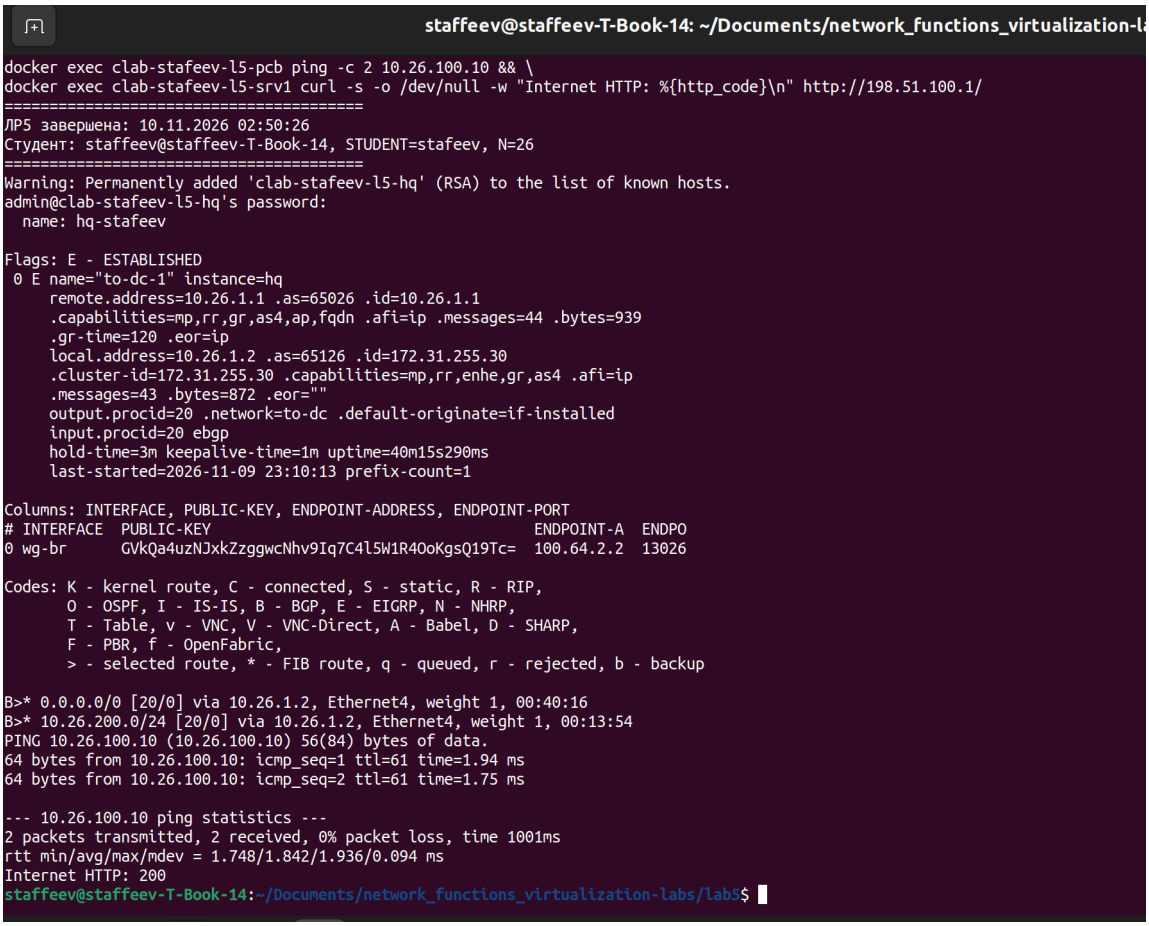


Рисунок 8.3 — Финальный артефакт

В ходе работы освоены принципы виртуализации сетевых функций (NFV) на примере построения мультивендорной корпоративной сети. В среде Containerlab развёрнута топология на базе MikroTik CHR (vCPE) и SONiC с FRR, настроено eBGP-взаимодействие между ЦОД и головным офисом, реализованы функции NAT и stateful межсетевого экрана. Организован защищённый туннель WireGuard между головным офисом и филиалом с анонсированием сети филиала в ЦОД через BGP. Управление маршрутизатором осуществлено через REST API, обеспечена полная воспроизводимость конфигурации как кода (Infrastructure as Code).

В результате получены практические навыки развёртывания виртуальных сетевых функций, настройки протокола BGP между разнородными

платформами, туннелирования WireGuard, программирования сетевых политик через REST API и автоматизации развёртывания через скрипты-оркестраторы. Цель работы выполнена.